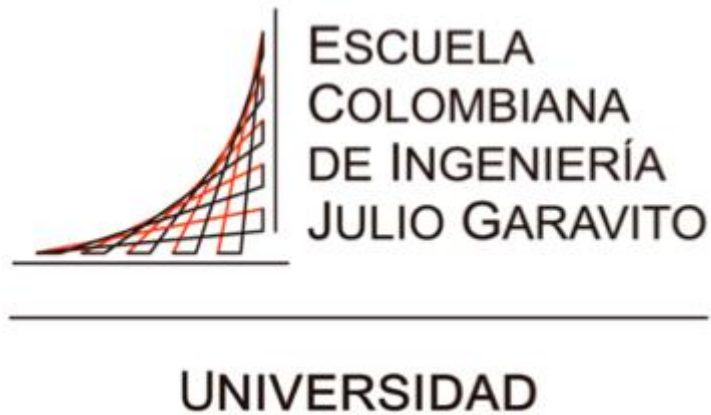




Nicolás Felipe Bernal Gallo

Juan Daniel Bogotá Fuentes

Desarrollo Orientada a objetos
DOPO

COLECCIONES Y PERSISTENCIA
PATRONES DE DISEÑO

PROFESORA: ANGIE TATIANA MEDINA GIL

1. Patrón Estado

Nombre

Estado

Problema

¿Cómo hacer que un objeto cambie su comportamiento cuando su estado interno cambia? ¿Cómo evitar el uso de múltiples condicionales (if-else) que verifican el estado del objeto y ejecutan diferentes comportamientos según ese estado?

Este problema surge cuando un objeto debe comportarse de manera diferente dependiendo de su estado actual, y el código termina lleno de condiciones que lo hacen difícil de mantener y extender.

Solución

Crear una clase abstracta o interfaz que defina el comportamiento común a todos los estados posibles. Luego, crear una clase concreta para cada estado específico, donde cada clase implementa el comportamiento particular de ese estado.

Estructura propuesta:

- Una interfaz o clase abstracta Estado que declara métodos para cada operación
- Clases concretas que implementan Estado, una por cada estado del objeto

Ejemplo en SilkRoad: Estado de las tiendas

En el proyecto, las tiendas pueden estar en diferentes estados: con tenges, sin tenges, en reabastecimiento, etc. Cada estado tiene comportamiento diferente.

Sin usar el patrón Estado:

Tendríamos múltiples if-else en cada método de las tiendas, verificando en qué estado está antes de ejecutar cada operación. Esto hace el código difícil de leer y mantener.

Usando el patrón Estado:

- El código de la tienda quedó limpio, sin muchas condiciones.
- Cada estado tiene su propia clase, así se entiende mucho más lo que hace.
- Si se quiere extender el código simplemente solo se crea una nueva clase.

De esta forma, cada estado encapsula su propio comportamiento y las transiciones entre estados son claras y fáciles de mantener.

2. Patrón Estrategia

Nombre

Estrategia

Problema

¿Cómo manejar situaciones donde existen múltiples algoritmos o comportamientos alternativos para realizar una misma tarea? ¿Cómo permitir que el cliente seleccione o cambie el algoritmo a utilizar sin modificar el código que lo usa?

Este problema aparece cuando tenemos diferentes formas de realizar una operación y queremos poder intercambiarlas fácilmente, evitando el uso de múltiples condicionales y facilitando la adición de nuevos algoritmos.

Solución

Definir una familia de algoritmos encapsulando cada uno en una clase separada, y hacer que estas clases sean intercambiables. Para esto, se crea una interfaz o clase abstracta que declara el método o métodos que implementan el algoritmo.

Es como tener diferentes “recetas” para hacer lo mismo. La idea es que el cliente pueda escoger qué algoritmo usar sin cambiar el código.

Estructura propuesta:

- Una interfaz que declara el método del algoritmo.
- Clases concretas que implementan su lógica, una para cada algoritmo.
- Polimorfismo prácticamente.

Ejemplo en SilkRoad

En el proyecto que hicimos Bernal y Bogotá usamos diferentes estrategias para decidir a qué tienda iría el robot, a la más cercana o a la más rentable.

Sin usar el patrón Estrategia:

Tendríamos múltiples if-else en los métodos de move() verificando qué tienda “robar”. Agregar un nuevo caso requeriría modificar este método.

Usando el patrón Estrategia:

- Se puede escoger el movimiento de preferencia.
- El método move() no está lleno de if's y else's.
- Cada estrategia tiene sus test's.
- El código es extensible.

De esta forma, podemos agregar nuevas estrategias de movimiento sin modificar el código existente, y el cliente puede cambiar la estrategia de movimiento según sus necesidades.

3. Patrón Propuesto: Validador en Cadena

Nombre

Validador en Cadena (Chain Validator)

Problema

En el desarrollo de aplicaciones, necesitamos validar datos de entrada que deben cumplir múltiples reglas. ¿Cómo permitir que las validaciones sean independientes, reutilizables y fáciles de agregar o remover?

Este problema surge cuando necesitamos validar datos de usuario, o parámetros de entrada, donde cada validación es independiente, pero todas son necesarias. Sin un patrón adecuado, terminamos con código repetitivo y difícil de mantener.

Solución

Crear una cadena de validadores donde cada validador es responsable de una regla específica. Cada validador decide si el dato es válido según su criterio y, si es válido, pasa el dato al siguiente validador en la cadena. Si algún validador encuentra un error, la cadena se detiene y retorna el error.

Es como cuando se pasa la solicitud a una cadena de personas y cada una decide si la maneja o la pasa a la siguiente.

Estructura propuesta:

- Una clase abstracta Validador con método validar(dato) y una referencia al siguiente validador
- Clases concretas que extienden Validador, una por cada regla de validación
- Cada validador concreto implementa su lógica de validación específica

Ejemplo en SilkRoad

Consideremos en el caso de que un robot se va a mover, necesitamos validar varias cosas: que tenga suficientes tenses, que la posición sea válida, etc. En vez de tener un método enorme (como los que nos ha corregido la profe), usamos ese tipo de cadenas.

Sin usar el patrón:

Tendríamos múltiples if o una serie de validaciones en un solo método, haciendo el código difícil de leer, mantener, reutilizar y no extensible.

Usando el patrón Validador en Cadena:

- Cada validador es independiente y reutilizable en diferentes contextos
- Fácil agregar o remover validaciones sin modificar código existente
- El orden de las validaciones puede cambiarse dinámicamente
- Código más limpio y fácil de mantener
- Facilita las pruebas unitarias de cada validación por separado